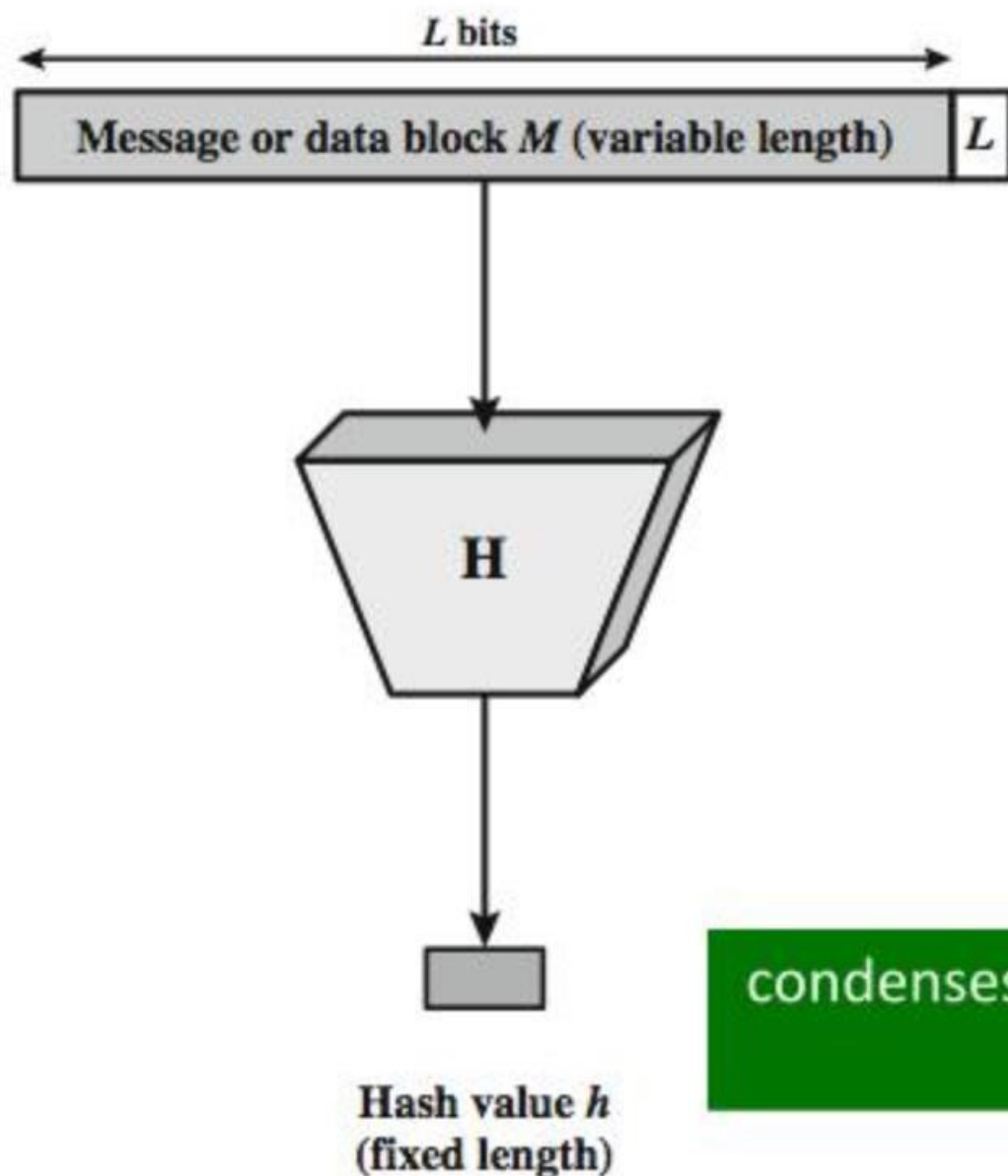


Hash Functions

Hash Function

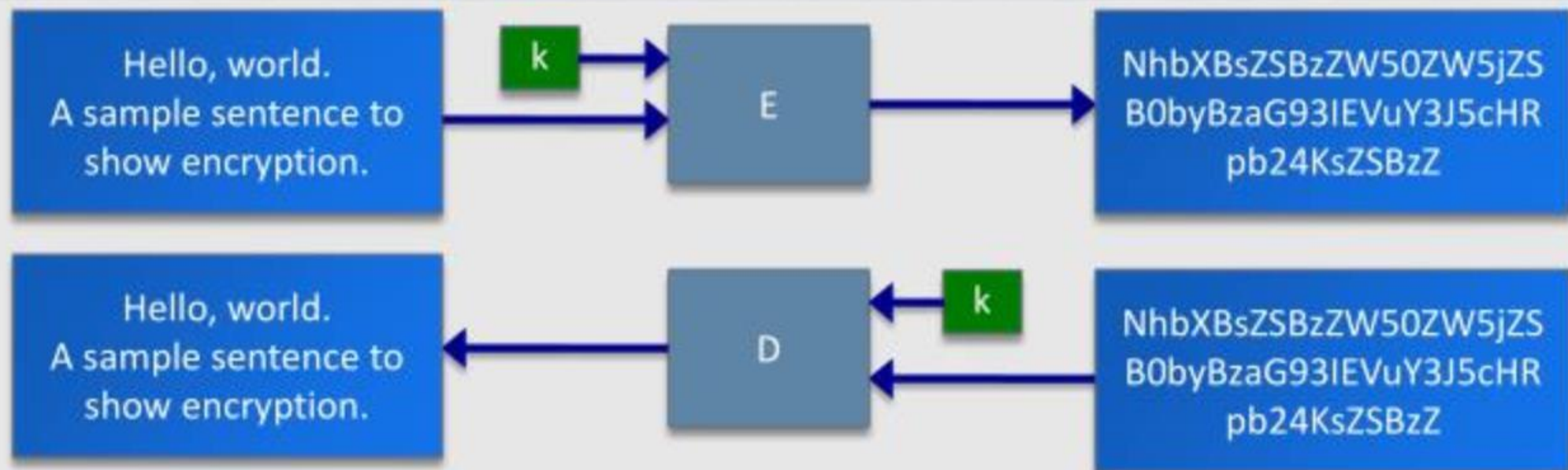


- ▶ The hash value represents concisely the longer message
 - ▶ may called the *message digest*
- ▶ A message digest is as a "digital fingerprint" of the original document

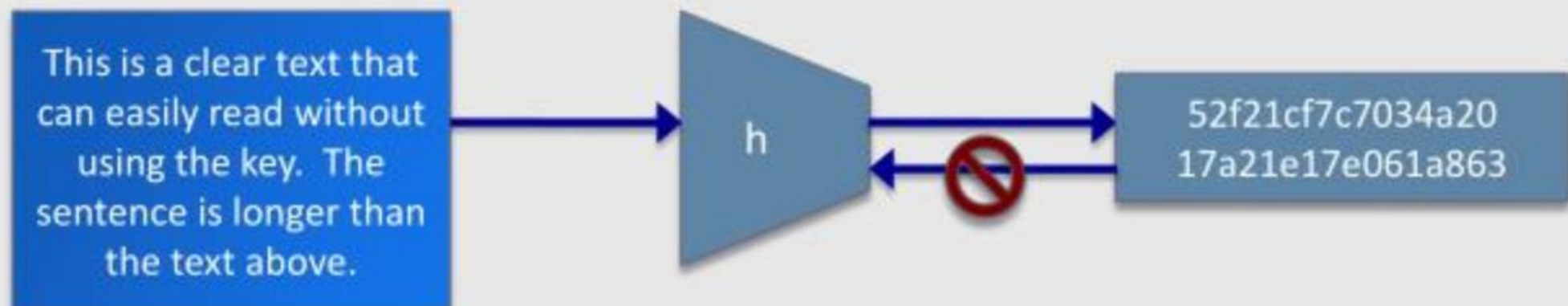
condenses arbitrary message to fixed size

$$h = H(M)$$

Hashing V.S. Encryption



- Encryption is two way, and requires a key to encrypt/decrypt

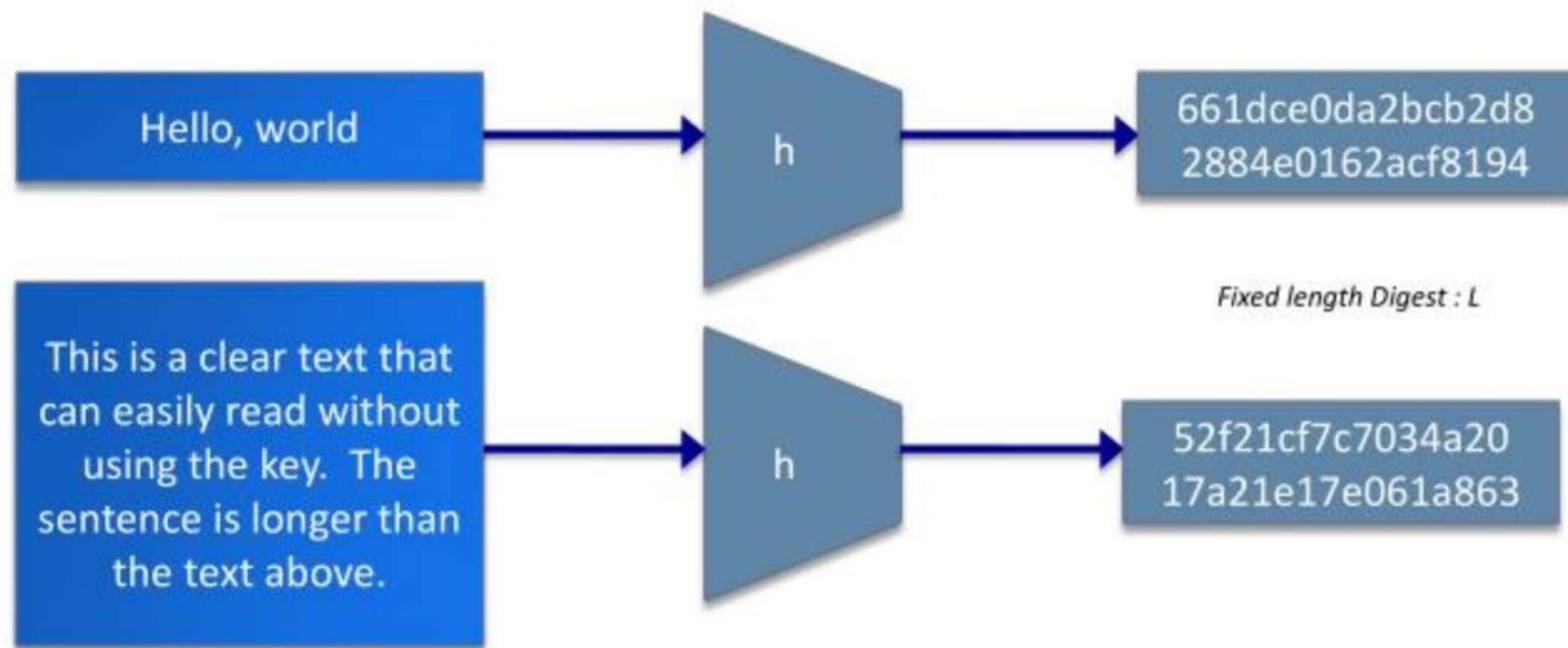


- Hashing is one-way. There is no 'de-hashing'

Hash Function Properties

- ▶ Arbitrary-length message to fixed-length digest
- ▶ Preimage resistant (**One-way property**)
- ▶ Second preimage resistant (**Weak collision resistant**)
- ▶ Collision resistant (**Strong collision resistance**)

Properties : Fixed length



- ▶ Arbitrary-length message to fixed-length digest

Preimage resistant

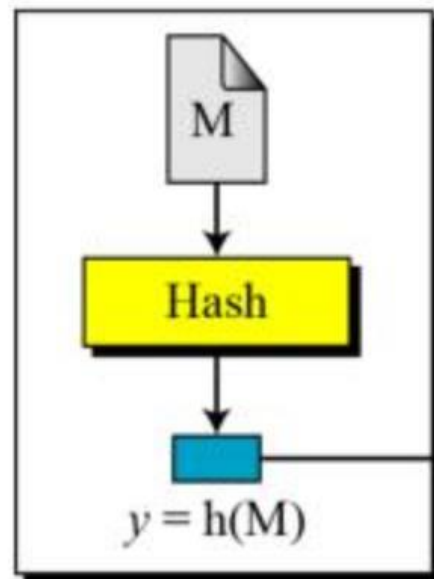
- ▶ This measures how difficult to devise a message which hashes to the known digest
- ▶ Roughly speaking, the hash function must be one-way.

Preimage Attack

Given: $y = h(M)$

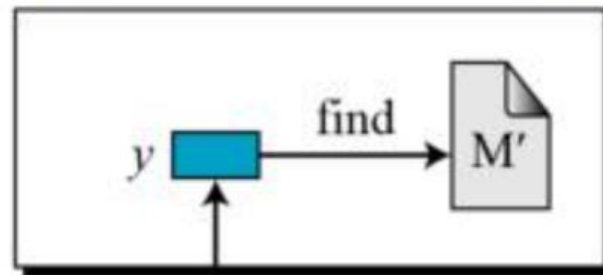
Find: M' such that $y = h(M')$

M: Message
Hash: Hash function
 $h(M)$: Digest



Alice

Given: y
Find: any M' such that
 $y = h(M')$



Eve

To Bob

Given only a message digest, can't find any message (or *preimage*) that generates that digest.

Second preimage resistant

- ▶ This measures how difficult to devise a message which hashes to the known digest and its message

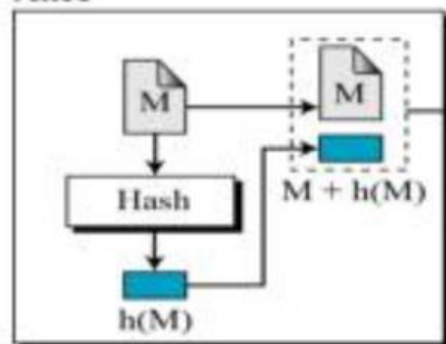
Second Preimage Attack

Given: M and $h(M)$

Find: $M' \neq M$ such that $h(M) = h(M')$

M : Message
Hash: Hash function
 $h(M)$: Digest

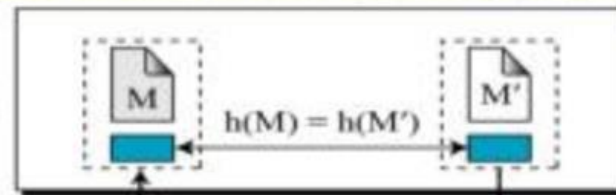
Alice



Given: M and $h(M)$

Find: M' such that $M \neq M'$, but $h(M) = h(M')$

Eve



To Bob

- ▶ Given one message, can't find another message that has the same message digest. An attack that finds a second message with the same message digest is a *second pre-image* attack.
 - ▶ It would be easy to forge new digital signatures from old signatures if the hash function used weren't second preimage resistant

Collision Resistant

Collision Attack

Given: none

Find: $M' \neq M$ such that $h(M) = h(M')$

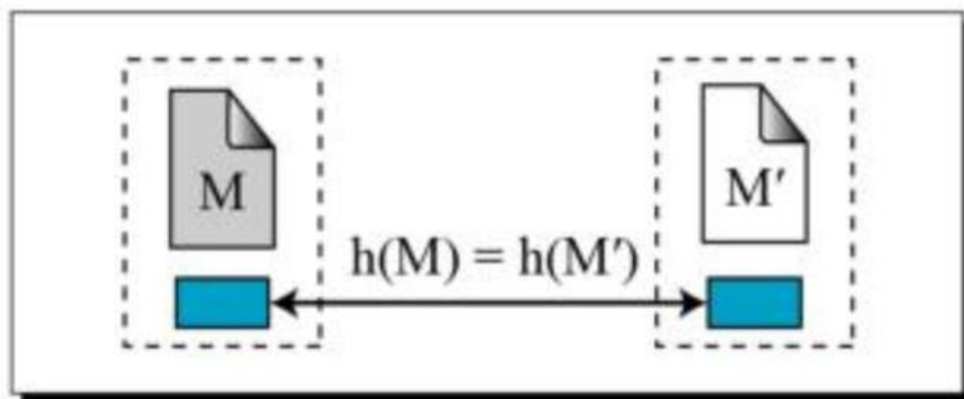
M: Message

Hash: Hash function

$h(M)$: Digest

Find: M and M' such that $M \neq M'$, but $h(M) = h(M')$

Eve



- ▶ Can't find any two different messages with the same message digest
 - ▶ Collision resistance implies second preimage resistance
 - ▶ Collisions, if we could find them, would give signatories a way to repudiate their signatures

Hash Functions Family

- ▶ **MD (Message Digest)**

- ▶ Designed by Ron Rivest
- ▶ Family: MD2, MD4, MD5

- ▶ **SHA (Secure Hash Algorithm)**

- ▶ Designed by NIST
- ▶ Family: SHA-0, SHA-1, and SHA-2
 - ▶ SHA-2: SHA-224, SHA-256, SHA-384, SHA-512
 - ▶ SHA-3: New standard in competition

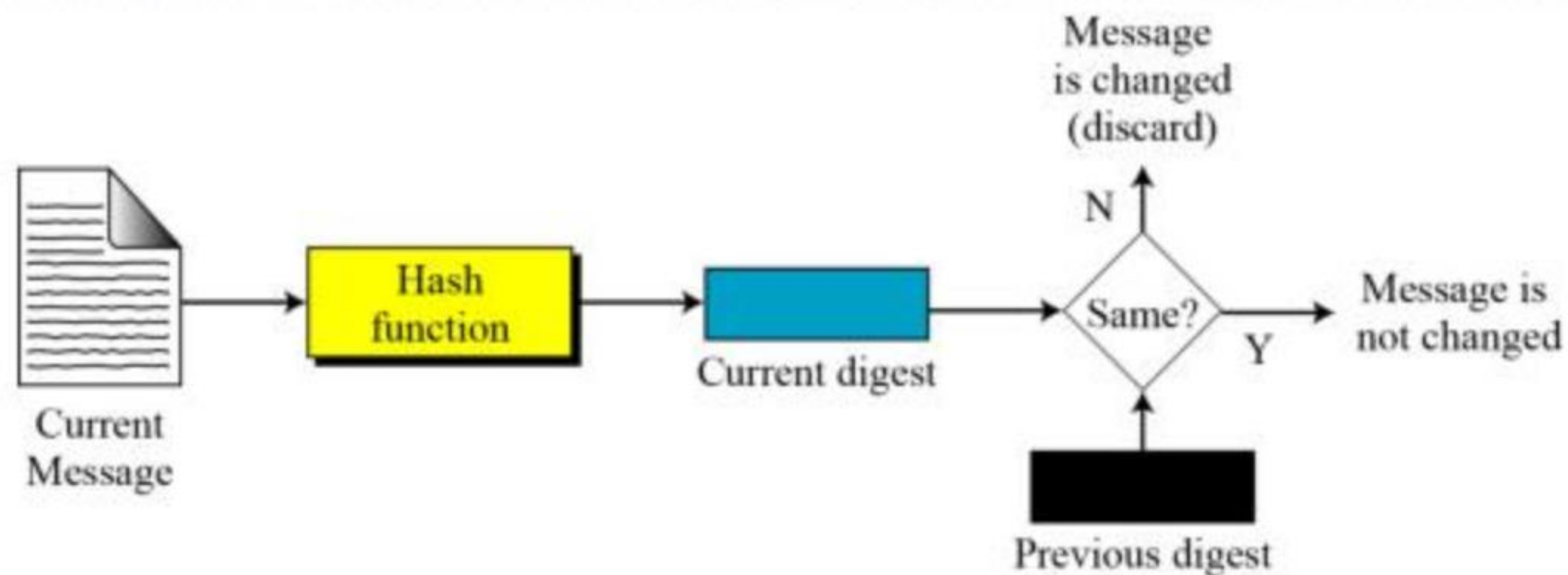
- ▶ **RIPEMD (Race Integrity Primitive Evaluation Message Digest)**

- ▶ Developed by Katholieke University Leuven Team
- ▶ Family : RIPEMD-128, RIPEMD-160, RIPEMD-256, RIPEMD-320,

Hash Function Applications

- ▶ Used Alone
 - ▶ Fingerprint -- file integrity verification, public key fingerprint
 - ▶ Password storage (one-way encryption)
- ▶ Combined with encryption functions
 - ▶ Message Authentication Code (MAC)
 - ▶ protects both a message's integrity as well as its authenticity
 - ▶ Digital signature
 - ▶ Ensuring Non-repudiation
 - ▶ Encrypt hash with private (signing) key and verify with public (verification) key
 - ▶

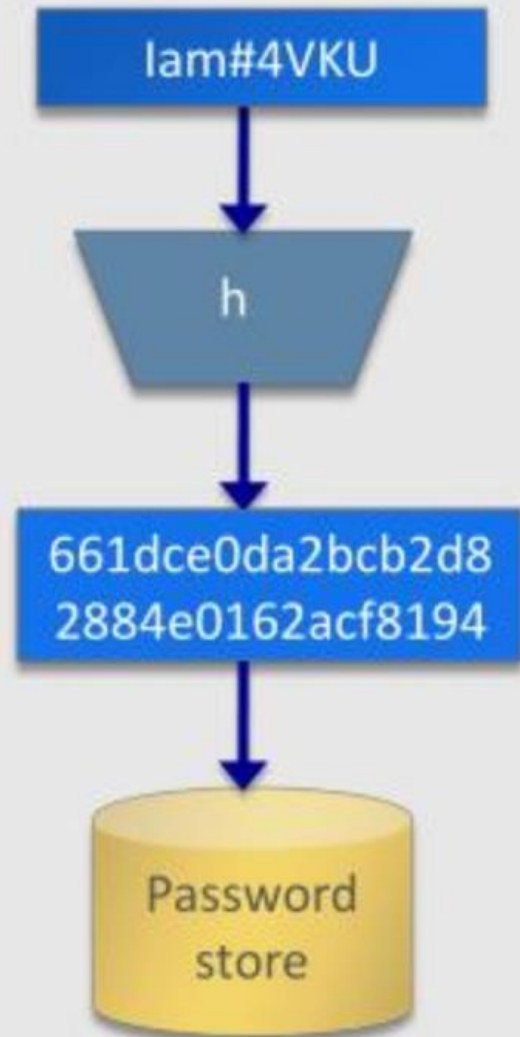
Integrity



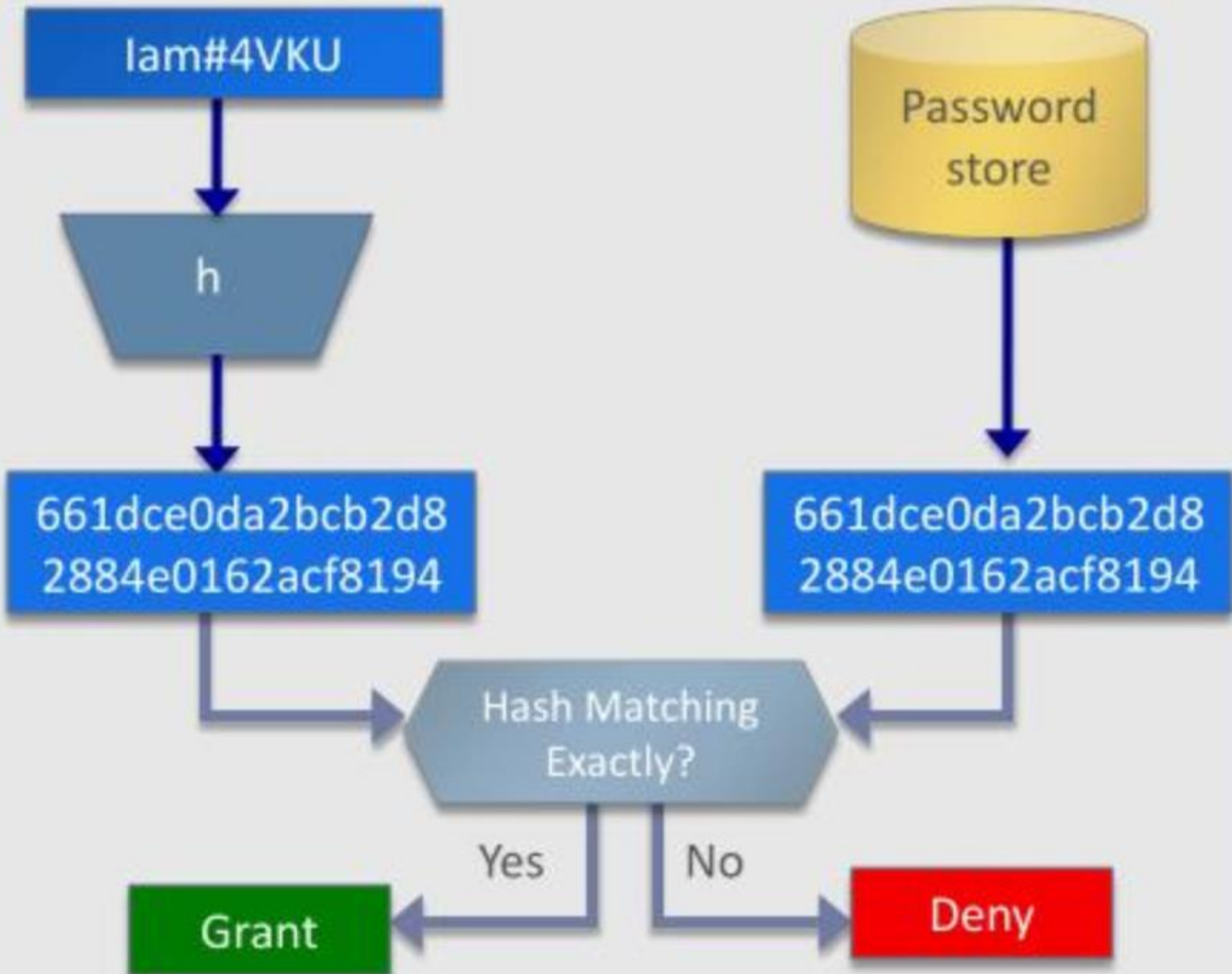
- ▶ to create a one-way password file
 - ▶ store hash of password not actual password
- ▶ for intrusion detection and virus detection
 - ▶ keep & check hash of files on system

Password Verification

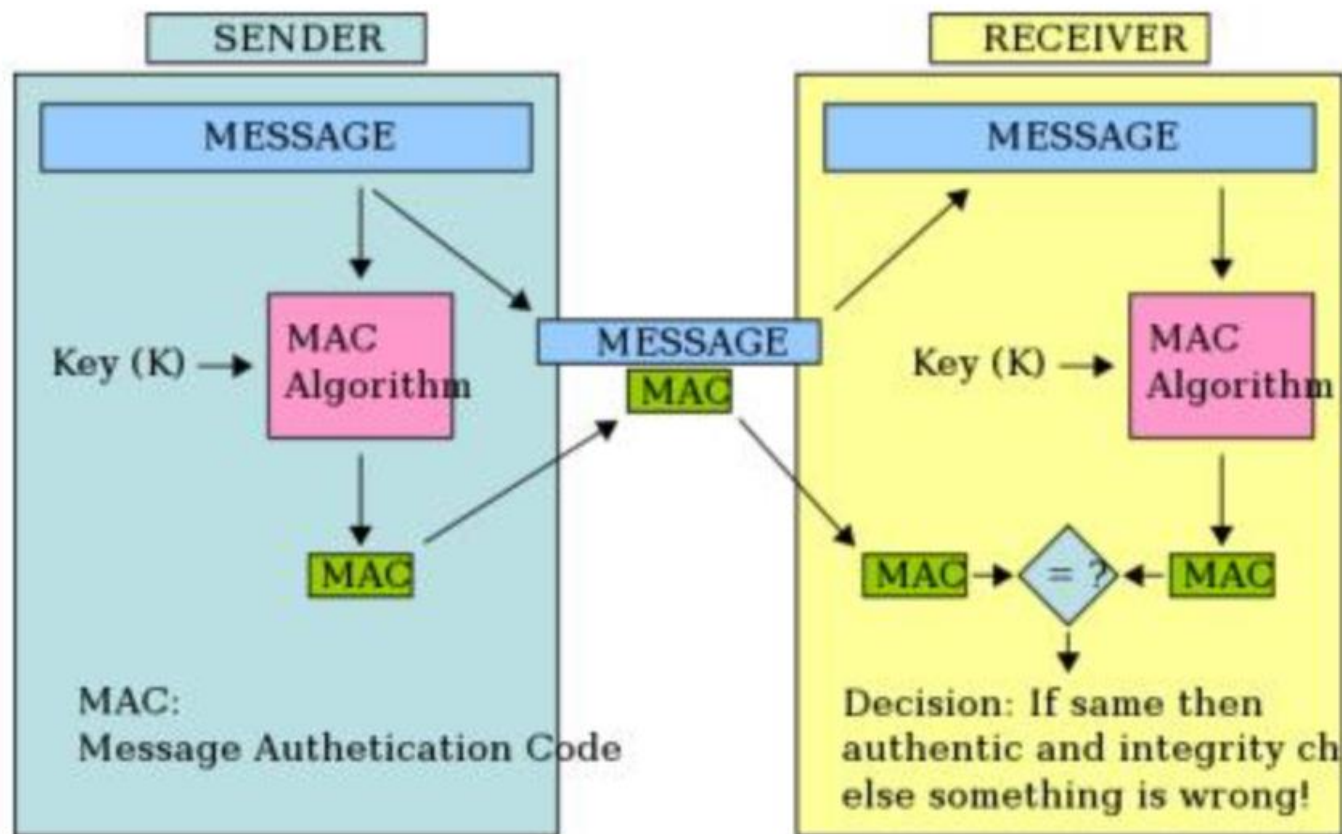
Store Hashing Password



Verification an input password against the stored hash



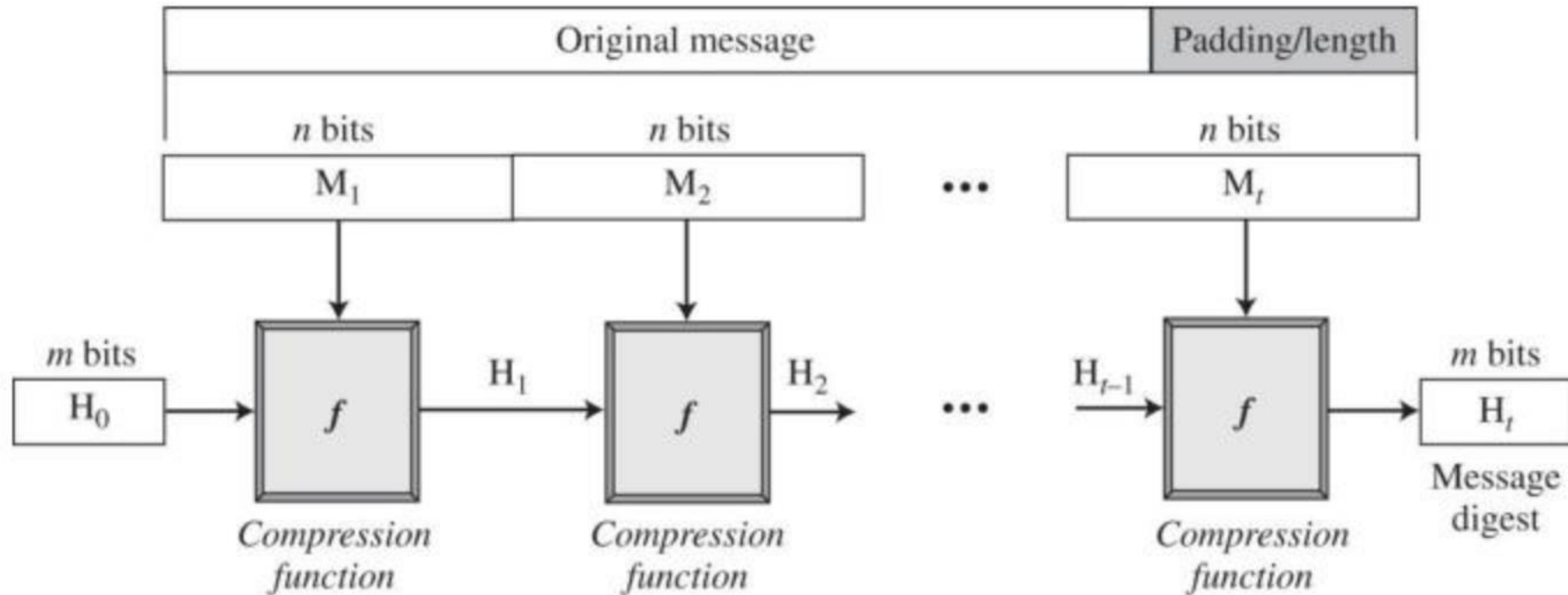
Authentication



- ▶ protects both a message's integrity as well as its authenticity , by allowing verifiers (who also possess the secret key) to detect any changes to the message content

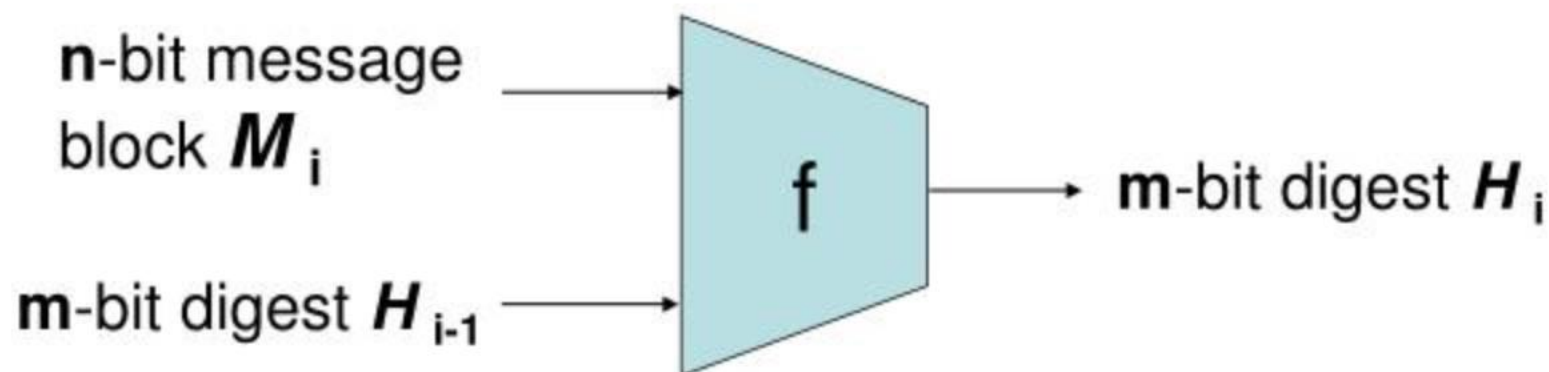
Iterated Hash Function

Merkle-Damgard scheme



Iterated Hash Function

- Compression function of form $\mathbf{h}(M_i, H_{i-1})$
 - M_i = i^{th} message block
 - H_{i-1} = previous message digest
 - H_0 = initial vector known to sender, recipient



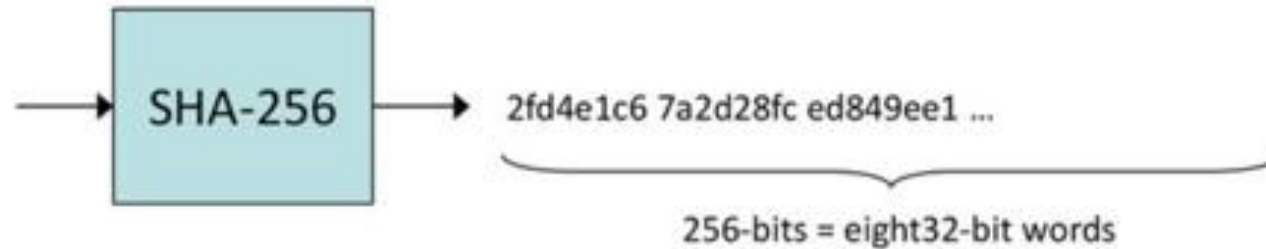
- If $\mathbf{f}$ is collision resistant, so is entire algorithm

SHA 256

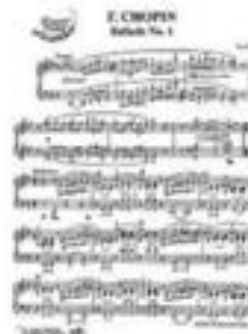
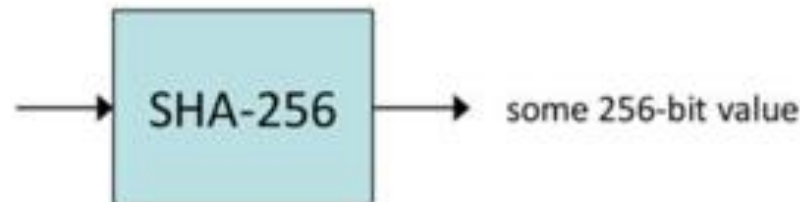
Secure Hash Algorithm

- Goal is to compute a unique hash value for any input “message”, where a “message” can be anything.
- SHA-256 (widely used) returns a 256-bit hash value (a.k.a. message digest or strong checksum)

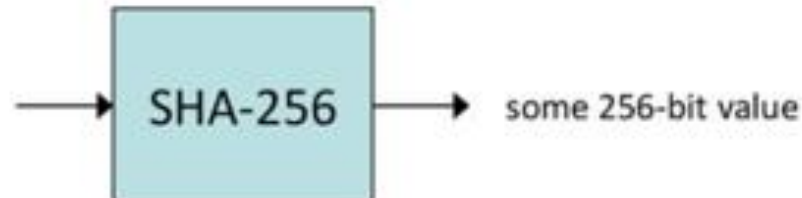
“The quick brown
fox jumps over the
lazy dog”



file: avatar.avi



file: chopin.mp3



SHA-256

- Just a small change, e.g. from “dog” to “cog”, will completely change the hash value

“The quick brown
fox jumps over the
lazy **dog**” → SHA-256 → 2fd4e1c6 7a2d28fc ed849ee1 ...

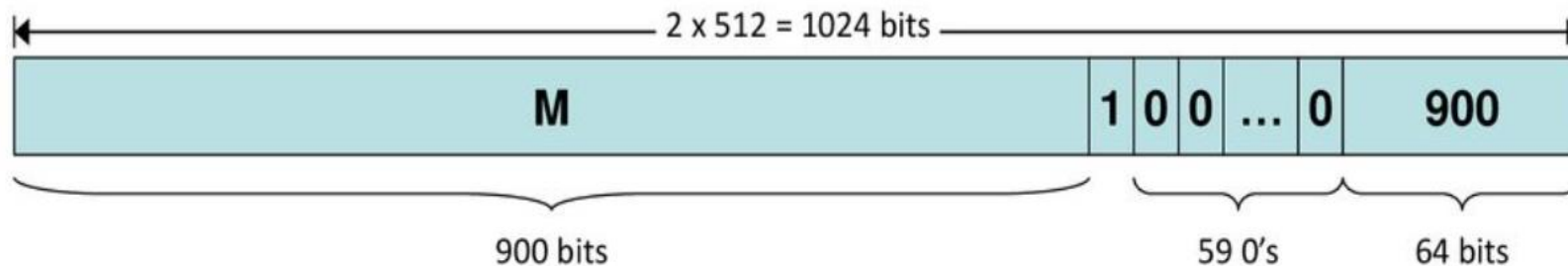
“The quick brown
fox jumps over the
lazy **cog**” → SHA-256 → de9f2c7f d25e1b3a fad3e85a ...

General Logic

- Input message must be $< 2^{64}$ bits
 - not really a problem
- Message is processed in 512-bit blocks sequentially
- Message digest is 256 bits

SHA-256 Algorithm

- Step 1: Padding bits
 - A **b**-bit message **M** is padded in the following manner:
 - Add a single “1” to the end of **M**
 - Then pad message with “0’s” until the length of message is congruent to 448, modulo 512 (which means pad with 0’s until message is 64-bits less than some multiple of 512).
- Step 2: Appending length as 64 bit unsigned
 - A 64-bit representation of **b** is appended to the result of Step 1.
 - The resulting message is a multiple of 512 bits
 - e.g. suppose $b = 900$



SHA-256 Algorithm

- Step 3: Buffer initiation – initialize message digest (MD) to these eight 32-bit words

$$H_0 = 6a09e667$$

$$H_1 = bb67ae85$$

$$H_2 = 3c6ef372$$

$$H_3 = a54ff53a$$

$$H_4 = 510e527f$$

$$H_5 = 9b05688c$$

$$H_6 = 1f83d9ab$$

$$H_7 = 5be0cd19$$

- These values are calculated from first 8 prime numbers (2,3,5,7,11, 13,17,19)
- Each value is the fraction part of the square root of corresponding prime number and converting into binary and keeping only the first 32 bits.

SHA-256 Algorithm

- Step 4: Processing of the message (the algorithm)
 - Divide message M into 512-bit blocks, $M_0, M_1, \dots M_j, \dots$
 - Process each M_j sequentially, one after the other
 - Input:
 - W_t : a 32-bit word from the message
 - K_t : a constant array
 - $H_0, H_1, H_2, H_3, H_4, H_5, H_6, H_7$: current MD
 - Output:
 - $H_0, H_1, H_2, H_3, H_4, H_5, H_6, H_7$: new MD

SHA-256 Algorithm

- Step 4: Cont'd
 - At the beginning of processing each M_j , initialize $(A, B, C, D, E, F, G, H) = (H_0, H_1, H_2, H_3, H_4, H_5, H_6, H_7)$
 - Then 64 processing rounds of 512-bit blocks
 - Each step t ($0 \leq t \leq 63$): Word expansion for W_t
 - If $t < 16$
 - $W_t = t^{\text{th}}$ 32-bit word of block M_j
 - If $16 \leq t \leq 63$
 - $s_0 = (W_{t-15} \text{ rightrotate } 7) \text{ xor } (W_{t-15} \text{ rightrotate } 18) \text{ xor } (W_{t-15} \text{ rightshift } 3)$
 - $s_1 = (W_{t-2} \text{ rightrotate } 17) \text{ xor } (W_{t-2} \text{ rightrotate } 19) \text{ xor } (W_{t-2} \text{ rightshift } 10)$
 - $W_t = W_{t-16} + s_0 + W_{t-7} + s_1$

SHA-256 Algorithm

- Step 4: Cont'd

- K_t constants

$K [0..63] = 0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,$
 $0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5, 0xd807aa98,$
 $0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe,$
 $0x9bdc06a7, 0xc19bf174, 0xe49b69c1, 0xefbe4786, 0x0fc19dc6,$
 $0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,$
 $0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3,$
 $0xd5a79147, 0x06ca6351, 0x14292967, 0x27b70a85, 0x2e1b2138,$
 $0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e,$
 $0x92722c85, 0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,$
 $0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070, 0x19a4c116,$
 $0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a,$
 $0x5b9cca4f, 0x682e6ff3, 0x748f82ee, 0x78a5636f, 0x84c87814,$
 $0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2$



There are 64 constants, K_0 to K_{63} , each of 32 bits. Like initialized hash values, these values are calculated from the first 64 prime numbers.

SHA-256 Algorithm

- Step 4: Cont'd

- Each step t ($0 \leq t \leq 63$):

$$S_0 = (A \text{ rightrotate } 2) \text{ xor } (A \text{ rightrotate } 13) \text{ xor } (A \text{ rightrotate } 22)$$

$$\text{maj} = (A \text{ and } B) \text{ xor } (A \text{ and } C) \text{ xor } (B \text{ and } C)$$

$$t_2 = S_0 + \text{maj}$$

$$S_1 = (E \text{ rightrotate } 6) \text{ xor } (E \text{ rightrotate } 11) \text{ xor } (E \text{ rightrotate } 25)$$

$$\text{ch} = (E \text{ and } F) \text{ xor } ((\text{not } E) \text{ and } G)$$

$$t_1 = H + S_1 + \text{ch} + K_t + W_t$$

$$(A, B, C, D, E, F, G, H) = (t_1 + t_2, A, B, C, D + t_1, E, F, G)$$

SHA-256 Algorithm

- Step 4: Cont'd
 - Finally, when all 64 steps have been processed, set

$$H_0 = H_0 + A$$

$$H_1 = H_1 + B$$

$$H_2 = H_2 + C$$

$$H_3 = H_3 + D$$

$$H_4 = H_4 + E$$

$$H_5 = H_5 + F$$

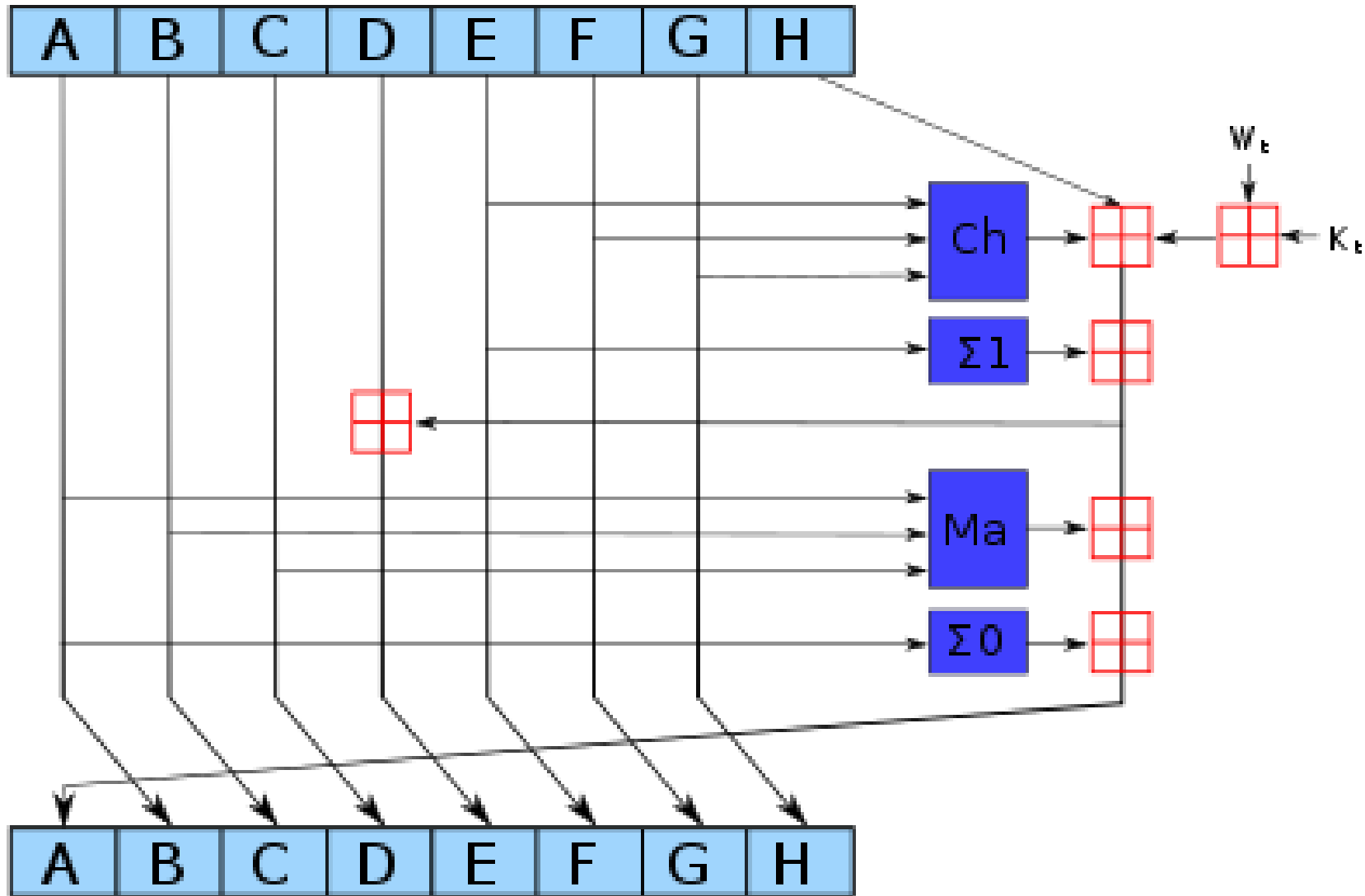
$$H_6 = H_6 + G$$

$$H_7 = H_7 + H$$

SHA-256 Algorithm

- Step 5: Output
 - When all M_j have been processed, the 256-bit hash of M is available in $H_0, H_1, H_2, H_3, H_4, H_5, H_6$, and H_7

Round Function in SHA 256



SHA-512

- Background:
 - Based on Merkle-Damgard scheme, Rivest MD5
- Ideas:
 - Large number of rounds (80) for maximum confusion
 - Heavy use of non-invertible functions
 - Combinations of rotation and XOR
 - Condition and majority functions
 - Appearance of “randomness” in initial vector
- Designed for efficiency
 - All operations are very fast in hardware

Overview

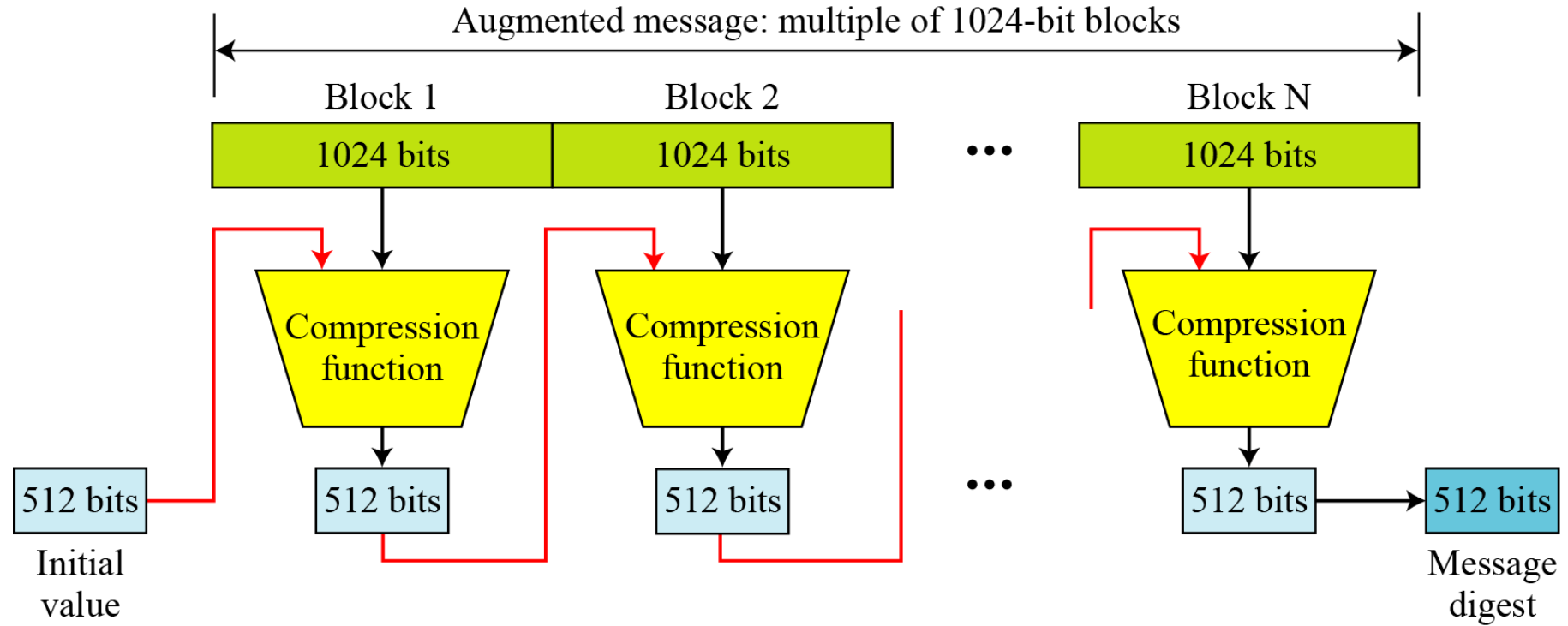
- ❑ developed by National Institute of Standards and Technology
- ❑ member of SHA-2 family
- ❑ latest version of Secure Hash Algorithm
- ❑ based on the Merkle-Damgard scheme
- ❑ maximum message size $2^{128}-1$ bits
- ❑ block size 1024 bits
- ❑ message digest size 512 bits
- ❑ number of rounds 80
- ❑ word size 64 bits

Processing of SHA-512

- appending padding and fixed 128 bit length field
- dividing the augmented message into blocks
- using a 64-bit word derived from the current message block
- using 8 constants based on square root of first 8 prime numbers (2-19)
- updating a 512-bit buffer
- using a round constant based on cube root of first 80 prime numbers (2-409)

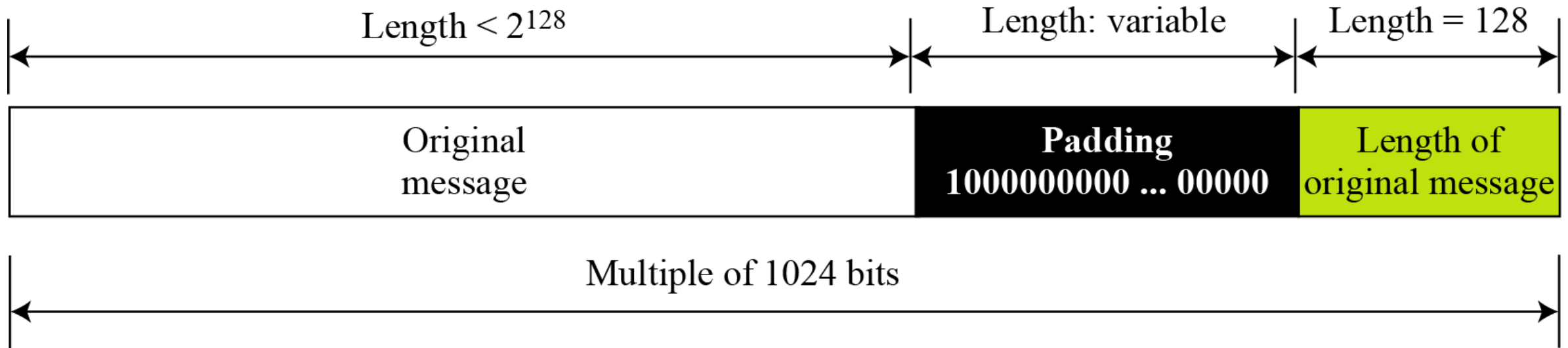


Message digest creation SHA-512





Padding and length field in SHA-512





What is the number of padding bits if the length of the original message is 2590 bits?

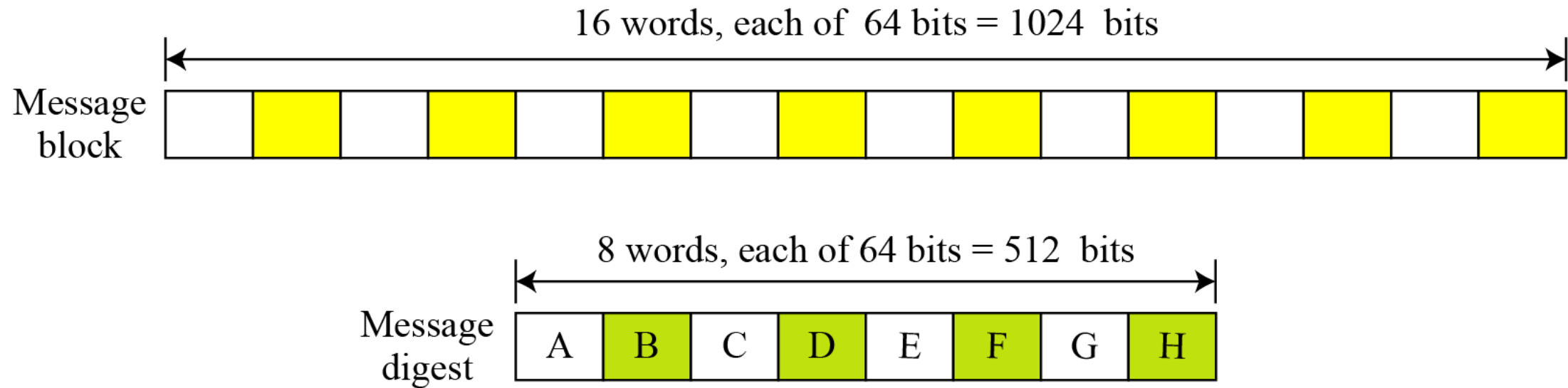
Solution

We can calculate the number of padding bits as follows:

$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

The padding consists of one 1 followed by 353 0's.

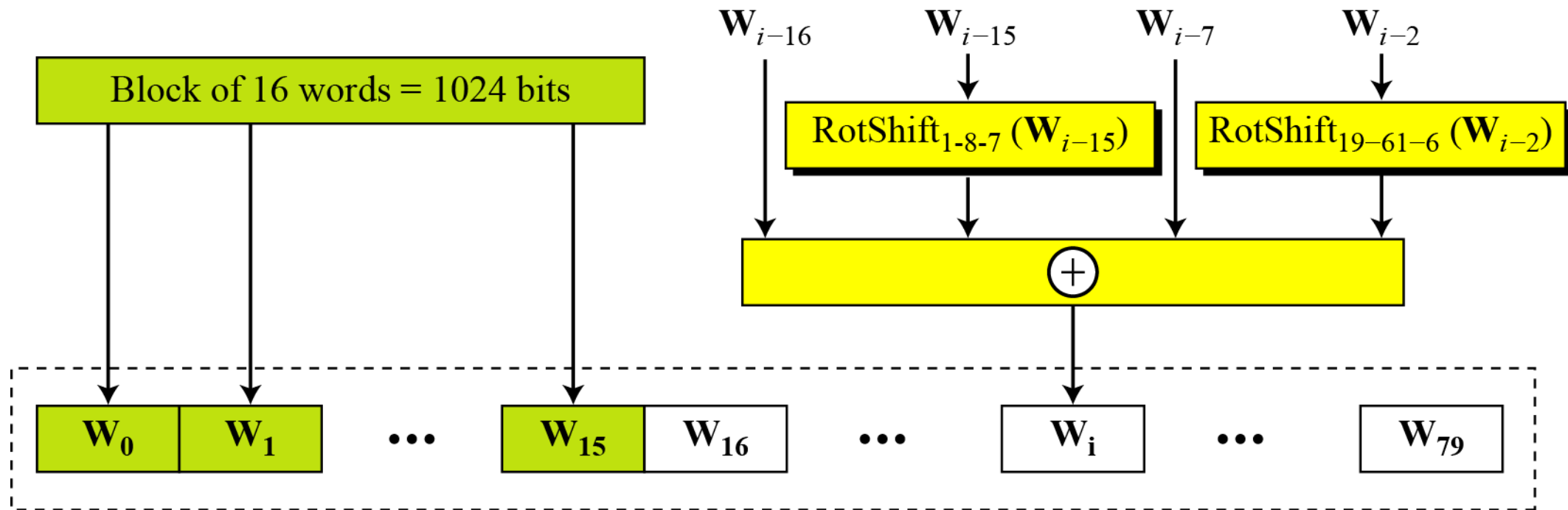
A message block and the digest as words





Word expansion in SHA-512

Block of 16 words are expanded to 80 words



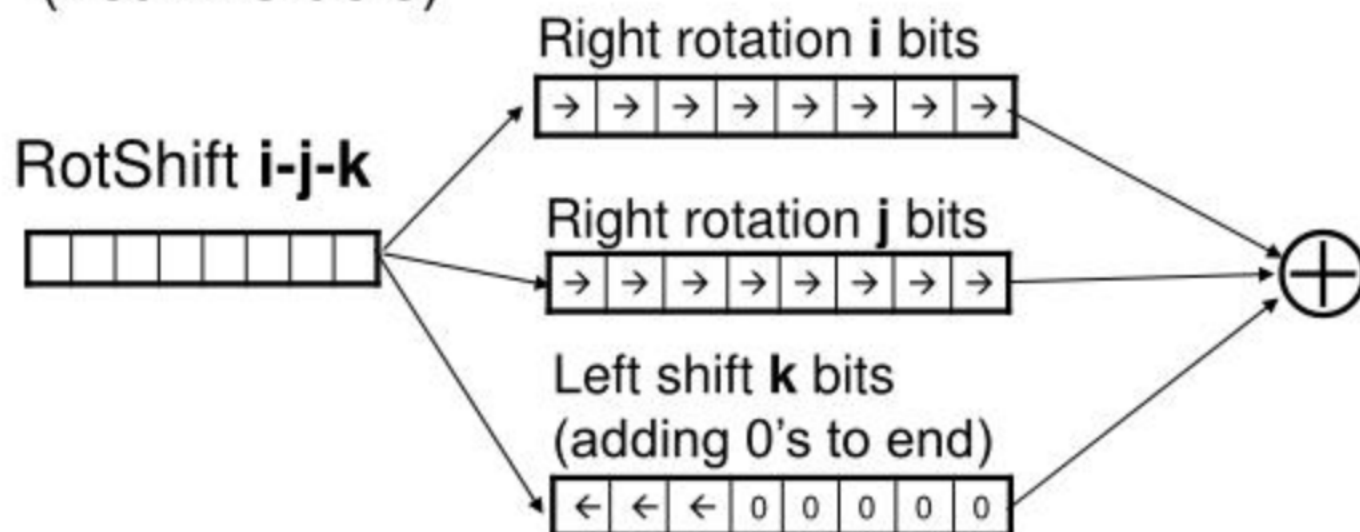
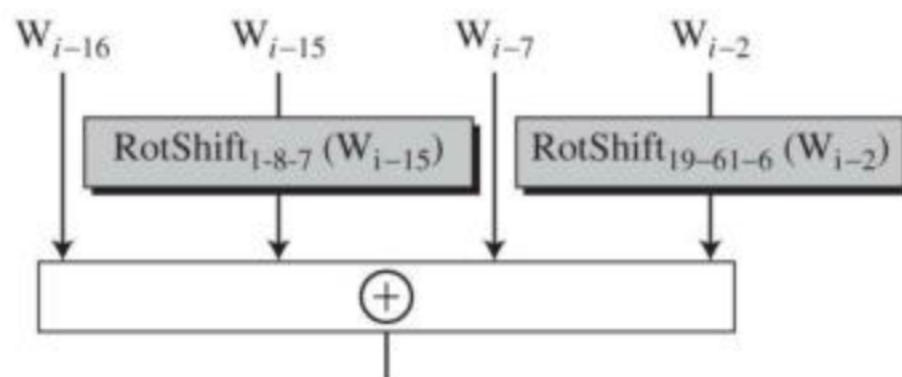
$\text{RotShift}_{l-m-n}(x): \text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x)$

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

$\text{ShL}_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

Word Expansion in SHA-512

- Each word function of previous 4 words
 - Combined with XOR
 - Confusion added with rotation and shifting (not invertible)





Show how W60 is made

•

Solution

Each word in the range W16 to W79 is made from four previously-made words. W60 is made as

$$W_{60} = W_{44} \oplus \text{RotShift}_{1-8-7}(W_{45}) \oplus W_{53} \oplus \text{RotShift}_{19-61-6}(W_{58})$$



Message Digest Initialization

Table 12.2 *Values of constants in message digest initialization of SHA-512*

<i>Buffer</i>	<i>Value (in hexadecimal)</i>	<i>Buffer</i>	<i>Value (in hexadecimal)</i>
A ₀	6A09E667F3BCC908	E ₀	510E527FADE682D1
B ₀	BB67AE8584CAA73B	F ₀	9B05688C2B3E6C1F
C ₀	3C6EF372EF94F828	G ₀	1F83D9ABFB41BD6B
D ₀	A54FE53A5F1D36F1	H ₀	5BE0CD19137E2179

- *These values are calculated from first 8 prime numbers (2,3,5,7,11, 13,17,19)*
- *Each value is the fraction part of the square root of the corresponding prime number and converting into binary and keeping only the first 64 bits*



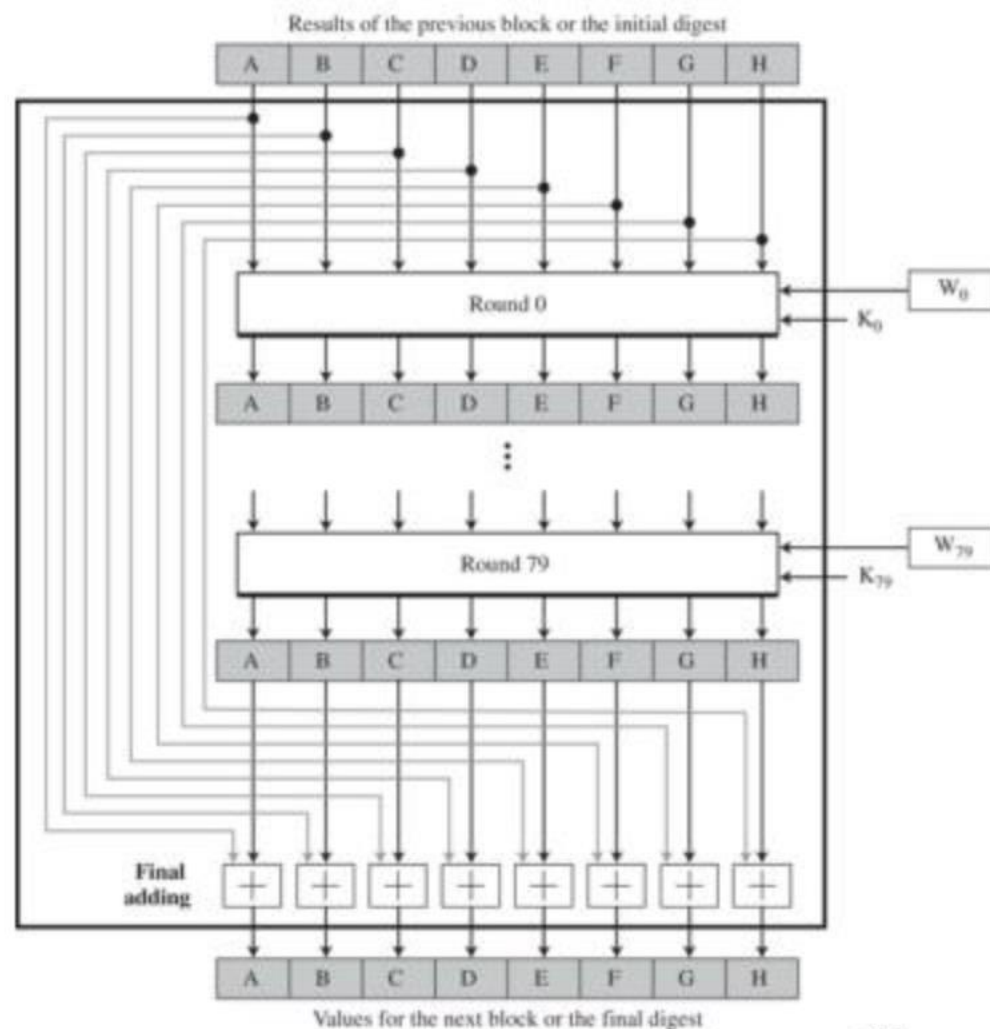
80 Constants used in Rounds of SHA 512

Table 12.3 *Eighty constants used for eighty rounds in SHA-512*

428A2F98D728AE22	7137449123EF65CD	B5C0FBCFEC4D3B2F	E9B5DBA58189DBBC
3956C25BF348B538	59F111F1B605D019	923F82A4AF194F9B	AB1C5ED5DA6D8118
D807AA98A3030242	12835B0145706FBE	243185BE4EE4B28C	550C7DC3D5FFB4E2
72BE5D74F27B896F	80DEB1FE3B1696B1	9BDC06A725C71235	C19BF174CF692694
E49B69C19EF14AD2	EFBE4786384F25E3	0FC19DC68B8CD5B5	240CA1CC77AC9C65
2DE92C6F592B0275	4A7484AA6EA6E483	5CB0A9DCBD41FBD4	76F988DA831153B5
983E5152EE66DFAB	A831C66D2DB43210	B00327C898FB213F	BF597FC7BEEF0EE4
C6E00BF33DA88FC2	D5A79147930AA725	06CA6351E003826F	142929670A0E6E70
27B70A8546D22FFC	2E1B21385C26C926	4D2C6DFC5AC42AED	53380D139D95B3DF
650A73548BAF63DE	766A0ABB3C77B2A8	81C2C92E47EDAE6	92722C851482353B
A2BFE8A14CF10364	A81A664BBC423001	C24B8B70D0F89791	C76C51A30654BE30
D192E819D6EF5218	D69906245565A910	F40E35855771202A	106AA07032BBD1B8
19A4C116B8D2D0C8	1E376C085141AB53	2748774CDF8EEB99	34B0BCB5E19B48A8
391C0CB3C5C95A63	4ED8AA4AE3418ACB	5B9CCA4F7763E373	682E6FF3D6B2B8A3
748F82EE5DEFB2FC	78A5636F43172F60	84C87814A1F0AB72	8CC702081A6439EC
90BEFFFA23631E28	A4506CEBDE82BDE9	BEF9A3F7B2C67915	C67178F2E372532B
CA273ECEEA26619C	D186B8C721C0C207	EADA7DD6CDE0EB1E	F57D4F7FEE6ED178
06F067AA72176FBA	0A637DC5A2C898A6	113F9804BEF90DAE	1B710B35131C471B
28DB77F523047D84	32CAAB7B40C72493	3C9EBE0A15C9BEBC	431D67C49C100D4C
4CC5D4BECB3E42B6	4597F299CFC657E2	5FCB6FAB3AD6FAEC	6C44198C4A475817

SHA-512 Compression Function

- 80 rounds
 - Each creates new “intermediate” message digest
- Final stage is sum (mod 2^{64}) of:
 - Initial round digest
 - Final round digest

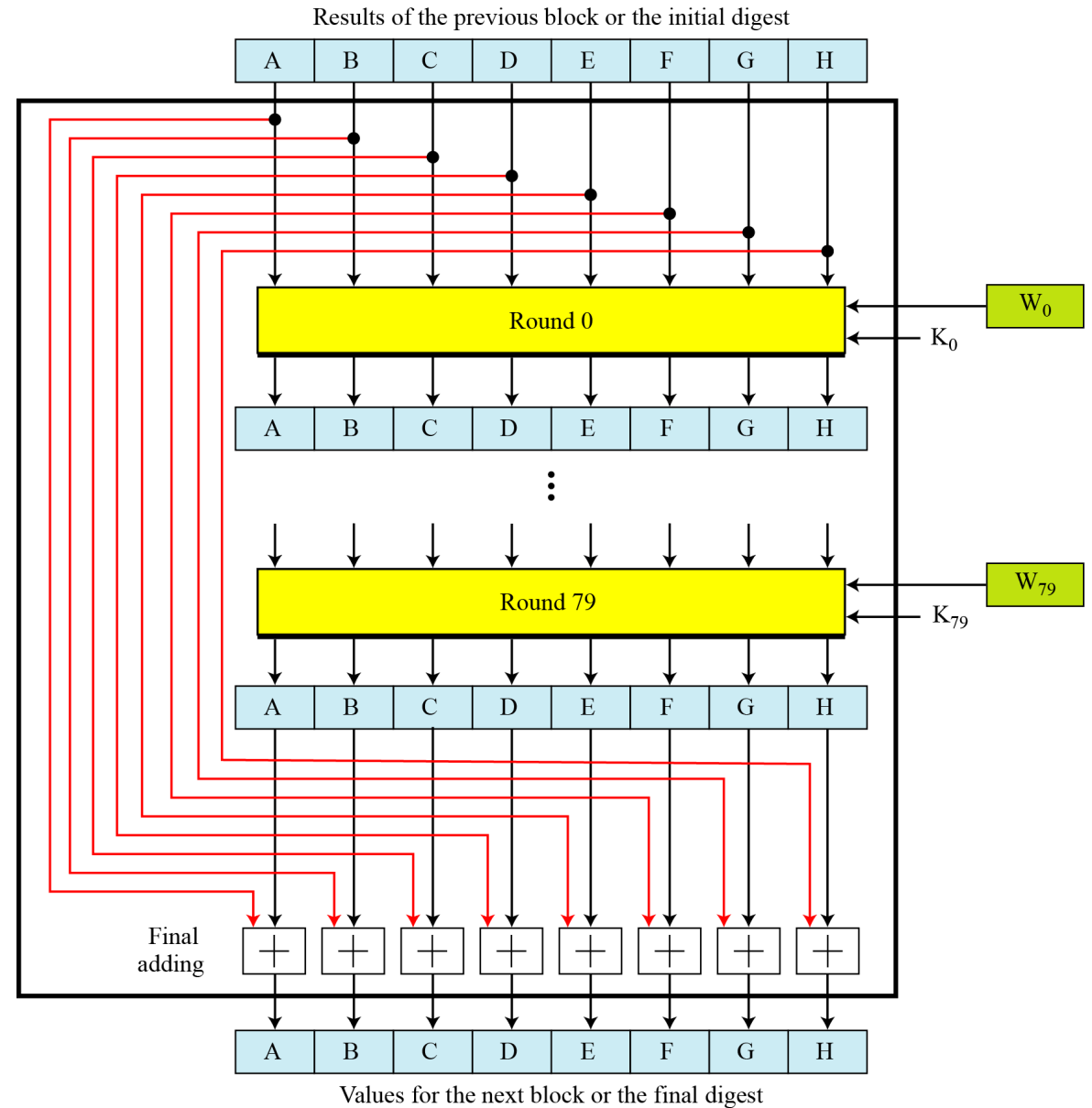




Compression Function in SHA-512

Each round is a function of

1. Previous Message Digest
2. Word W_i
3. Round Constants K_i created from fractional parts of cube root of first 80 prime numbers





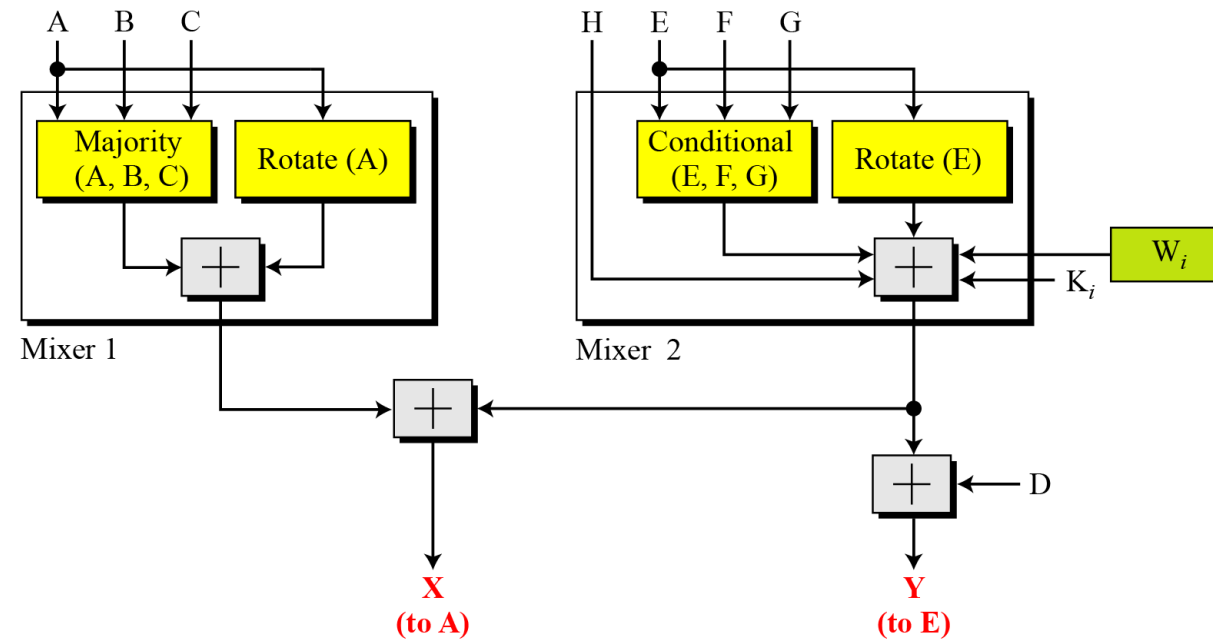
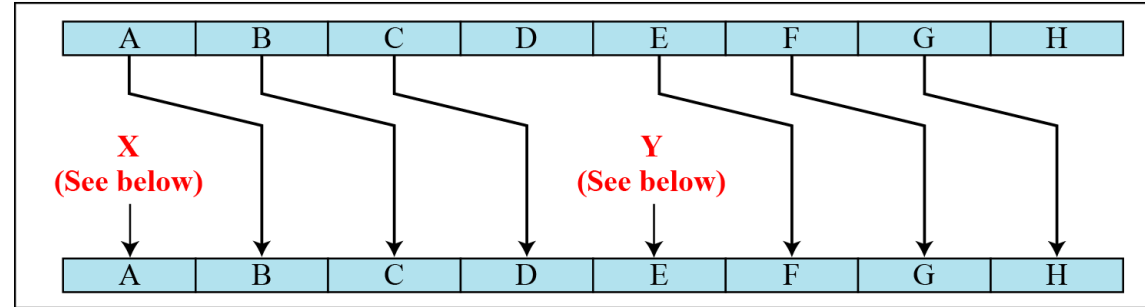
*There are 80 constants, K_0 to K_{79} , each of 64 bits. Similar
These values are calculated from the first 80 prime
numbers (2, 3,..., 409). For example, the 80th prime is
409, with the cubic root $(409)^{1/3} = 7.42291412044$.
Converting this number to binary with only 64 bits in the
fraction part, we get*

$$(111.0110\ 1100\ 0100\ 0100\ \dots\ 0111)_2 \rightarrow (7.6C44198C4A475817)_{16}$$

The fraction part: $(6C44198C4A475817)_{16}$

Structure of each round in SHA-512

Round



Majority (x, y, z)

$$(x \text{ AND } y) \oplus (y \text{ AND } z) \oplus (z \text{ AND } x)$$

Rotate (x)

$$\text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$$

Conditional (x, y, z)

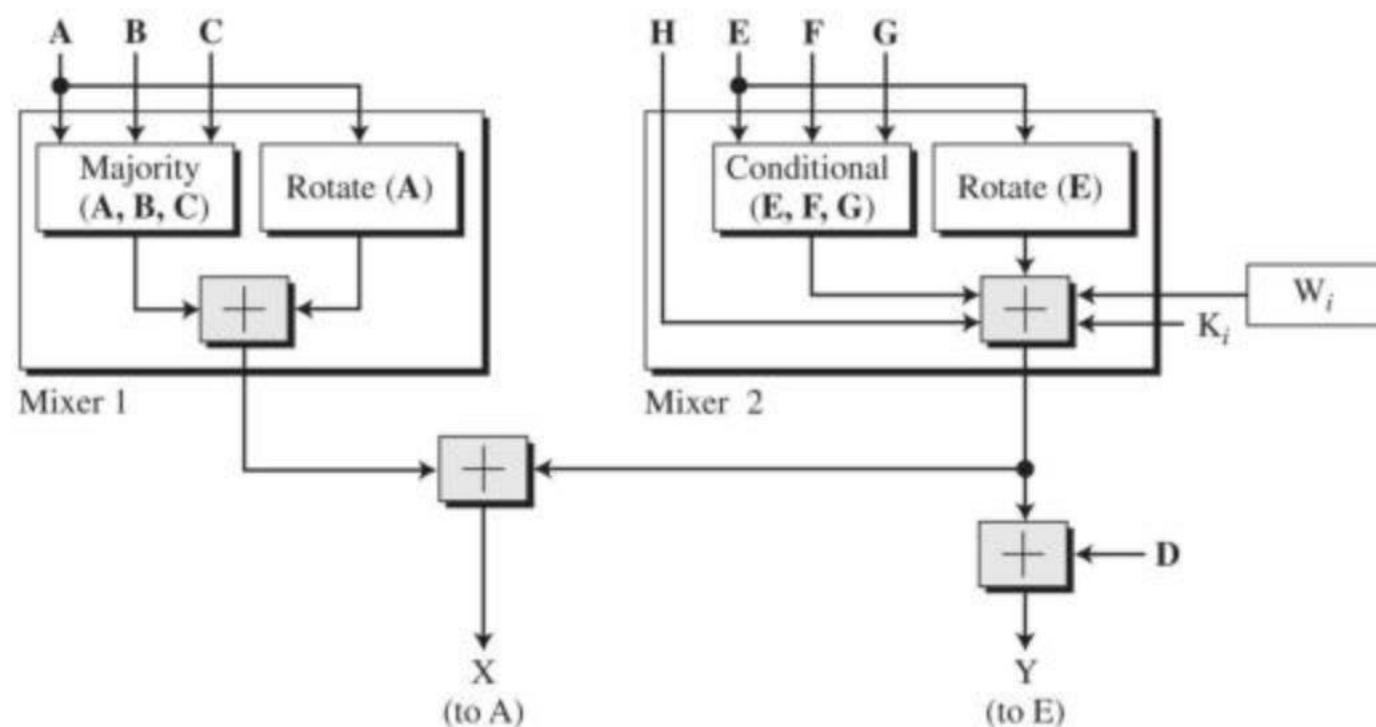
$$(x \text{ AND } y) \oplus (\text{NOT } x \text{ AND } z)$$

$\oplus$ addition modulo 2^{64}

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

SHA-512 Round Structure

- New blocks A and E created as function of:
 - All previous blocks A – G
 - Round word W_i and round key K_i using addition mod 2^{64}





Majority Function

$$(A_j \text{ AND } B_j) \oplus (B_j \text{ AND } C_j) \oplus (C_j \text{ AND } A_j)$$

Conditional Function

$$(E_j \text{ AND } F_j) \oplus (\text{NOT } E_j \text{ AND } G_j)$$

Rotate Functions

$$\text{Rotate (A): RotR}_{28}(A) \oplus \text{RotR}_{34}(A) \oplus \text{RotR}_{29}(A)$$

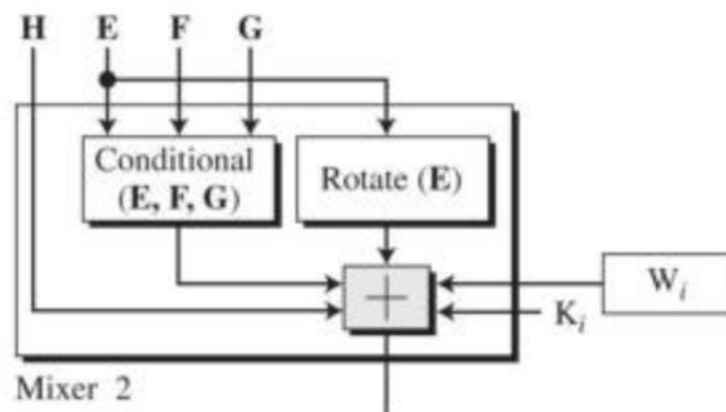
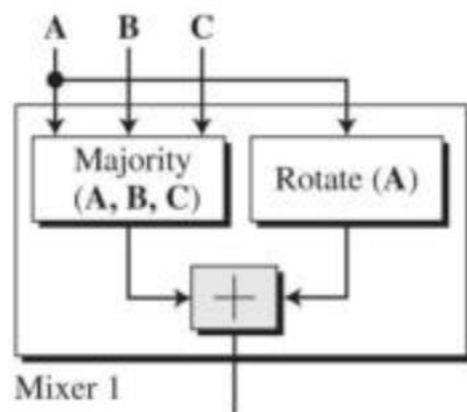
$$\text{Rotate (E): RotR}_{28}(E) \oplus \text{RotR}_{34}(E) \oplus \text{RotR}_{29}(E)$$

SHA-512 Rotation Function

- Next value of **A**, **E** based on previous value
- Rotate right by 28, 34, and 39 bits
- Combine with XOR to mix up bits

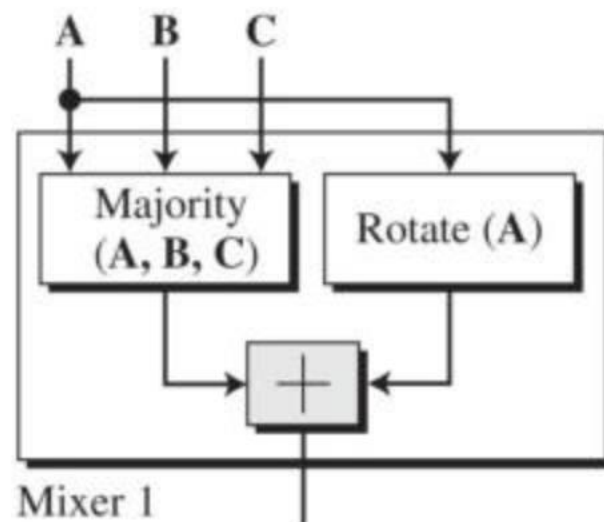
Rotate (x)

$$RotR_{28}(x) \oplus RotR_{34}(x) \oplus RotR_{39}(x)$$



SHA-512 Majority Function

- Majority function of (A, B, C):
 - i^{th} bit of result = **1** if at least 2 of i^{th} bits of A, B, C = 1
0 otherwise
 - Example: **A** = 11001010
B = 01101001
C = 10011101
majority = 11001001
 - Idea: No way to reconstruct
A, B, C from **majority**





Majority Function

We apply the Majority function on buffers A, B, and C. If the leftmost hexadecimal digits of these buffers are 0x7, 0xA, and 0xE, respectively, what is the leftmost digit of the result?

Solution

The digits in binary are 0111, 1010, and 1110.

- a. The first bits are 0, 1, and 1. The majority is 1.**
- b. The second bits are 1, 0, and 1. The majority is 1.**
- c. The third bits are 1, 1, and 1. The majority is 1.**
- d. The fourth bits are 1, 0, and 0. The majority is 0.**

The result is 1110, or 0xE in hexadecimal.

SHA-512 Conditional Function

- Conditional function of (E, F, G):
 - i^{th} bit of result = i^{th} bit of **F** if i^{th} bit of **E** = 1
= i^{th} bit of **G** otherwise

- Like “If **E** then **F** else **G**”

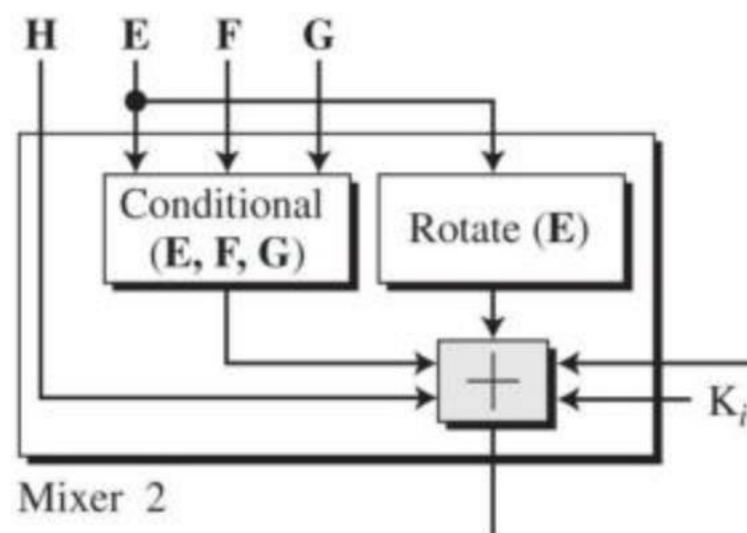
- Example: **E** = 11001010

F = 01101001

G = 10011101

Conditional = 01011101

- Idea: No way to reconstruct
E, F, G from **conditional**





We apply the Conditional function on E, F, and G buffers. If the leftmost hexadecimal digits of these buffers are 0x9, 0xA, and 0xF respectively, what is the leftmost digit of the result?

Solution

The digits in binary are 1001, 1010, and 1111.

- a. The first bits are 1, 1, and 1. The result is F_1 , which is 1.**
- b. The second bits are 0, 0, and 1. The result is G_2 , which is 1.**
- c. The third bits are 0, 1, and 1. The result is G_3 , which is 1.**
- d. The fourth bits are 1, 0, and 1. The result is F_4 , which is 0.**

The result is 1110, or 0xE in hexadecimal.